Relatório de Bugs

Aplicação: Automation Exercise — https://www.automationexercise.com

DOCUMENTO	Relatório de defeitos e propostas de correção
VERSÃO	1.1
DATA	16/10/2026
AUTOR	Angelo Roque — Análise de Qualidade
FERRAMENTAS	REA 6.5.0 (captura de cenários de navegador e evidências), Chromium 1243 (Playwright), cURL, Cypress 16.1.1
STATUS GERAL	11 defeitos confirmados e reproduzíveis — 4 alta, 5 média, 2 baixa
HISTÓRICO	1.0 (14/10/2026): BUG-01 a BUG-05 · 1.1 (16/10/2026): BUG-06 a BUG-11

1. Sumário executivo

Este relatório apresenta os defeitos identificados na aplicação web Automation Exercise durante duas rodadas de análise exploratória apoiada por engenharia reversa da aplicação em execução. Foram confirmados onze defeitos, todos reproduzíveis e cobertos por testes automatizados em Cypress que falham enquanto o defeito persistir.

Os defeitos mais graves envolvem a conta do usuário e o atendimento. Uma simples requisição `GET` para `/delete_account` apaga a conta de quem estiver logado, sem confirmação nem proteção contra CSRF (BUG-06). A API pública devolve nome, endereço e data de nascimento de qualquer e-mail cadastrado, sem autenticação (BUG-07). E o formulário *Contact Us* exibe "Success!" sem enviar a mensagem a lugar nenhum (BUG-08).

No carrinho, a quantidade não é validada: valores negativos geram totais negativos (BUG-01), zero gera itens sem valor (BUG-02) e valores vazios ou não inteiros derrubam o servidor com a página de debug do Django (BUG-09). A newsletter e as avaliações de produto também exibem mensagens de sucesso sem enviar nada (BUG-10 e BUG-11). Os demais defeitos envolvem recursos carregados de forma insegura (Mixed Content) e incoerências no contrato HTTP da API.

ID	Título	Área	Severidade	Prioridade
BUG-01	Carrinho aceita quantidade negativa	Carrinho / Back-end	**Alta**	P1
BUG-02	Carrinho aceita quantidade zero	Carrinho / Back-end	**Média**	P2
BUG-03	Fontes carregadas via HTTP em página HTTPS (Mixed Content)	Front-end / CSS	**Baixa**	P3
BUG-04	API retorna HTTP 200 para respostas de erro	API REST	**Média**	P2
BUG-05	API serve JSON com Content-Type text/html	API REST	**Baixa**	P3
BUG-06	Conta é excluída por um GET sem confirmação nem token CSRF	Conta / Segurança	**Alta**	P1
BUG-07	API expõe dados pessoais sem autenticação	API REST / Privacidade	**Alta**	P1
BUG-08	Contact Us confirma o envio, mas não envia nada	Atendimento / Front-end	**Alta**	P1
BUG-09	Quantidade vazia ou não inteira gera erro 500	Carrinho / Back-end	**Média**	P2
BUG-10	Assinatura da newsletter não é registrada	Newsletter / Front-end	**Média**	P2
BUG-11	Avaliação de produto não é enviada	Produto / Front-end	**Média**	P2

2. Escopo e metodologia

2.1 Escopo analisado

Rodada 1 (14/10/2026, BUG-01 a BUG-05):

- Página inicial (carregamento de recursos, console e rede);
- Página de detalhes do produto e fluxo de adição ao carrinho;
- Página do carrinho (`/view_cart`);
- API pública documentada em `/api_list` .

Rodada 2 (16/10/2026, BUG-06 a BUG-11):

- Conta logada: login, logout, exclusão de conta e consulta de dados pela API;
- Pedido completo (checkout e pagamento com dados de teste) e estado do carrinho depois do pedido e do logout;
- Formulários *Contact Us*, newsletter e avaliação de produto;
- Entradas inválidas na quantidade (vazio, decimal, notação científica, valores muito altos);
- Coerência de nome e preço entre listagem, página de detalhe e API (34 produtos) e das contagens por marca.

Fora do escopo: upload de arquivo no *Contact Us*, meios de pagamento reais e testes de carga ou de segurança ofensivos. Os testes de segurança se limitaram a observar o comportamento com contas de teste criadas para isso.

2.2 Metodologia

1. **Captura de cenários reais com a REA:** o comando `rea capture-browser-scenario` executou cenários declarativos em um Chromium isolado (perfil temporário, sem dados do usuário) e registrou como evidência os eventos de console, os erros de página, as requisições e respostas de rede e a árvore de acessibilidade.
2. **Análise estática dos recursos:** inspeção de `/static/js/cart.js` , `/static/js/subscription.js` , `/static/css/main.css` , dos scripts embutidos nas páginas e da marcação HTML, para localizar a causa raiz.
3. **Verificação da API:** requisições com cURL comparando o status HTTP real, o campo `responseCode` do corpo e os cabeçalhos.
4. **Diálogos JavaScript:** a REA não trata `confirm()` . Para o *Contact Us*, um script do Playwright aceitou o diálogo e registrou todas as requisições feitas depois do envio.
5. **Reprodução:** na rodada 2, cada achado foi reproduzido pelo menos duas vezes; os testes de conta usaram contas descartáveis, criadas pela API e removidas ao final.
6. **Confirmação automatizada:** cada defeito ganhou um teste Cypress que descreve o comportamento correto. Todos os testes falharam pelas asserções que descrevem os defeitos, o que os confirma.

2.3 Ambiente de teste

Sistema operacional	Ubuntu 26.04.1 LTS (linux-x64)
Navegador	Chromium 1243 (Playwright), headless, 1280×720, locale en-US, fuso UTC
Node.js	v24.14.0
Cypress	16.1.1 (navegador Electron)
REA	6.4.0 na rodada 1; 6.5.0 na rodada 2
Tecnologia observada no servidor	Django (inferido pelo token `csrfmiddlewaretoken` e pela página de erro), Phusion Passenger 6.2.0, atrás do Cloudflare

2.4 Critérios de classificação

Severidade	Critério
Alta	Afeta a integridade de dados financeiros ou de usuários, regras de negócio centrais ou a confidencialidade de dados pessoais; pode ser explorado pelo usuário ou por terceiros.
Média	Comportamento incorreto com impacto funcional ou de integração, sem perda financeira ou de dados direta.
Baixa	Impacto visual, de conformidade ou de boas práticas, com contorno simples.

3. Detalhamento dos defeitos

BUG-01 — Carrinho aceita quantidade negativa

Severidade / Prioridade	**Alta** / P1
Componente	`/product_details/{id}`, `/static/js/cart.js`, endpoint `GET /add_to_cart/{id}`
Reprodutibilidade	100% (determinística)
Pré-condição	Nenhuma (usuário anônimo, carrinho vazio)

PASSOS PARA REPRODUZIR

1. Acessar `https://www.automationexercise.com/product_details/1` (produto Blue Top, Rs. 500).
2. No campo *Quantity*, apagar o valor e digitar `-3`.
3. Clicar em *Add to cart*.
4. No modal, clicar em *View Cart*.

RESULTADO ESPERADO

A quantidade deve ser rejeitada, com mensagem de validação (mínimo 1). Nenhum item deve ser adicionado ao carrinho.

RESULTADO OBTIDO

O item é adicionado e o carrinho exibe: `Blue Top | Rs. 500 | -3 | Rs. -1500`.

EVIDÊNCIAS

- **Rede (REA):** requisição `GET https://www.automationexercise.com/add_to_cart/1?quantity=-3` aceita pelo servidor.
- **Árvore de acessibilidade do carrinho (REA):** `row "Product Image Blue Top Women > Tops Rs. 500 -3 Rs. -1500"`.
- **Cypress:** `AssertionError: expected -3 to be above 0`.

ANÁLISE DE CAUSA RAIZ

O campo é declarado como `<input type="number" id="quantity" min="1">`. O atributo `min`, porém, só é aplicado pelo navegador quando um formulário é enviado. O botão é `<button type="button" class="cart">`, e o envio é feito por AJAX em `cart.js`, que lê o valor bruto e o envia sem nenhuma validação:

```
$('.cart').on('click', function() {
    var product = document.getElementById("product_id").value;
    var quantity = document.getElementById("quantity").value;
    $.ajax({
        url: "/add_to_cart/" + product,
        data: {'quantity': quantity},
        ...
```

O servidor também aceita o valor sem validar o intervalo. A falha existe, portanto, em duas camadas. A camada decisiva é o back-end, porque a validação no navegador pode ser contornada chamando a URL diretamente.

IMPACTO

- Total do carrinho negativo ou reduzido indevidamente (combinando itens positivos e negativos);
- Risco de manipulação de preço no checkout e de inconsistência em estoque, relatórios e faturamento;
- Experiência do usuário degradada e falta de confiabilidade nos valores exibidos.

SOLUÇÃO PROPOSTA

1. Back-end (obrigatório): validar a quantidade na view, como inteiro dentro de um intervalo permitido, e responder com erro 400 quando o valor for inválido. Exemplo em Django:

```
from django.http import JsonResponse

MIN_QUANTITY = 1
MAX_QUANTITY = 99

def add_to_cart(request, product_id):
    try:
        quantity = int(request.GET.get("quantity", MIN_QUANTITY))
    except (TypeError, ValueError):
        return JsonResponse({"message": "Quantidade inválida."}, status=400)

    if not MIN_QUANTITY ⩽ quantity ⩽ MAX_QUANTITY:
        return JsonResponse(
            {"message": f"A quantidade deve estar entre {MIN_QUANTITY} e {MAX_QUANTITY}."},
            status=400,
        )

    # ... lógica existente de adição ao carrinho ...
```

2. Modelo de dados (defesa em profundidade): impedir que valores inválidos sejam persistidos, mesmo que outra rota grave no carrinho:

```
from django.db import models

class CartItem(models.Model):
    quantity = models.PositiveIntegerField()

    class Meta:
        constraints = [
            models.CheckConstraint(
                condition=models.Q(quantity__gte=1),
                name="cart_item_quantity_gte_1",
            ),
        ]
```

3. Front-end (experiência do usuário): usar a validação nativa do navegador antes de enviar e tratar o erro do servidor:

```
$('.cart').on('click', function () {
    var input = document.getElementById("quantity");
    var quantity = Number(input.value);
    if (!input.checkValidity() || !Number.isInteger(quantity) || quantity < 1) {
        input.reportValidity();
        return;
    }
    $.ajax({
        url: "/add_to_cart/" + document.getElementById("product_id").value,
        data: { quantity: quantity },
        success: function () {
            $('#cartModal').removeClass('fade').addClass('show');
        },
        error: function (xhr) {
            alert(xhr.responseJSON?.message || "Não foi possível adicionar o produto.");
        }
    });
});
```

4. Saneamento: executar uma rotina única para remover ou corrigir itens de carrinho já gravados com quantidade menor ou igual a zero.

CRITÉRIO DE ACEITE

- `GET /add_to_cart/1?quantity=-3` retorna HTTP 400 e não altera o carrinho;
- A interface exibe a mensagem de validação e não abre o modal de sucesso;
- O teste Cypress “[BUG-01] não deve aceitar quantidade negativa (-3)” passa.

BUG-02 — Carrinho aceita quantidade zero

Severidade / Prioridade	**Média** / P2
Componente	Mesmo do BUG-01
Reprodutibilidade	100%

PASSOS PARA REPRODUZIR

Os mesmos do BUG-01, informando `0` no campo *Quantity*.

RESULTADO ESPERADO

A quantidade deve ser rejeitada; nenhum item deve ser adicionado.

RESULTADO OBTIDO

O carrinho exibe uma linha sem valor: `Blue Top | Rs. 500 | 0 | Rs. 0`.

EVIDÊNCIAS

- **REA:** árvore de acessibilidade com `row "Product Image Blue Top Women > Tops Rs. 500 0 Rs. 0"`.
- **Cypress:** `AssertionError: expected 0 to be above 0`.

CAUSA RAIZ E SOLUÇÃO

A causa raiz é a mesma do BUG-01: falta validar o limite inferior. A correção proposta para o BUG-01 (intervalo 1..99 no back-end, restrição no banco e validação no front-end) resolve os dois defeitos, e também o BUG-09. Recomenda-se corrigi-los na mesma entrega.

IMPACTO

Itens "fantasmas" no carrinho, que confundem o usuário e podem gerar pedidos com linhas sem valor e falhas em integrações de estoque e logística.

BUG-03 — Fontes carregadas via HTTP em página HTTPS (Mixed Content)

Severidade / Prioridade	**Baixa** / P3
Componente	`/static/css/main.css`, linhas 5 a 7
Reprodutibilidade	100% em navegadores modernos

PASSOS PARA REPRODUZIR

1. Abrir `https://www.automationexercise.com/` com as ferramentas de desenvolvedor abertas (aba Console).
2. Observar os erros de *Mixed Content*.

RESULTADO ESPERADO

Todos os recursos são carregados via HTTPS e as fontes Roboto, Open Sans e Abel são aplicadas.

RESULTADO OBTIDO

O navegador bloqueia três folhas de estilo do Google Fonts, e a página é exibida com fontes substitutas.

EVIDÊNCIAS

- **Console (REA):** `Mixed Content: The page at 'https://www.automationexercise.com/' was loaded over HTTPS, but requested an insecure stylesheet 'http://fonts.googleapis.com/css?family=Abel'. This request has been blocked; the content must be served over HTTPS.` (mesma mensagem para Roboto e Open Sans);
- **Rede (REA):** três eventos `request-failed` para `http://fonts.googleapis.com/...`;
- **Código-fonte:**

```
@import url(http://fonts.googleapis.com/css?family=Roboto:400,300,400italic,500,700,100);
@import url(http://fonts.googleapis.com/css?family=Open+Sans:400,800,300,600,700);
@import url(http://fonts.googleapis.com/css?family=Abel);
```

- **Cypress:** `@import http:// em .../static/css/main.css: expected [ Array(3) ] to be empty`.

CAUSA RAIZ

As URLs foram escritas com o esquema `http://`, provavelmente desde antes da migração do site para HTTPS. Os navegadores bloqueiam esse tipo de recurso em páginas seguras.

IMPACTO

Identidade visual diferente da projetada, erros no console que poluem a monitoração e prejuízo em auditorias de qualidade e segurança (Lighthouse, por exemplo). Além disso, os `@import` encadeados atrasam a renderização.

SOLUÇÃO PROPOSTA

1. Correção mínima: trocar o esquema para HTTPS em `main.css`:

```
@import url(https://fonts.googleapis.com/css?family=Roboto:400,300,400italic,500,700,100);
@import url(https://fonts.googleapis.com/css?family=Open+Sans:400,800,300,600,700);
@import url(https://fonts.googleapis.com/css?family=Abel);
```

2. Correção recomendada (melhor desempenho): remover os `@import` do CSS e carregar as fontes em uma única requisição no `<head>` do template base:

```
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link rel="stylesheet"
      href="https://fonts.googleapis.com/css2?
family=Abel&family=Open+Sans:wght@300;400;600;700;800&family=Roboto:ital,wght@0,100;0,300;0,400;0,500;0,700;1,400&display=swap">
```

3. Prevenção: adicionar o cabeçalho `Content-Security-Policy: upgrade-insecure-requests`, que faz o navegador atualizar automaticamente para HTTPS os recursos referenciados com HTTP.

CRITÉRIO DE ACEITE

Nenhum erro de *Mixed Content* no console da página inicial; fontes aplicadas; teste Cypress do BUG-03 passando.

BUG-04 — API retorna HTTP 200 para respostas de erro

Severidade / Prioridade	**Média** / P2
Componente	Endpoints `/api/*`
Reprodutibilidade	100%

PASSOS PARA REPRODUZIR

```
curl -i -X POST https://www.automationexercise.com/api/productsList
```

RESULTADO ESPERADO

Status HTTP coerente com o erro (`405 Method Not Allowed` , com o cabeçalho `Allow: GET`).

RESULTADO OBTIDO

Status HTTP `200 OK` , com o erro apenas no corpo: `{"responseCode": 405, "message": "This request method is not supported."}` .

EVIDÊNCIAS

Requisição	Status HTTP real	`responseCode` no corpo
`POST /api/productsList`	200	405
`PUT /api/brandsList`	200	405
`DELETE /api/verifyLogin`	200	405
`POST /api/searchProduct` (sem parâmetro)	200	400
`POST /api/verifyLogin` (parâmetros em query string)	200	400

Cypress: `status HTTP real: expected 200 to equal 405` e `expected 200 to equal 400` .

CAUSA RAIZ

As views da API devolvem o código de erro como dado no corpo JSON, mas não o definem como status da resposta HTTP, que fica no padrão 200. A página `/api_list` documenta “Response Code: 405” sem esclarecer que o código não é o status HTTP.

IMPACTO

- Clientes HTTP, gateways, CDNs, ferramentas de monitoramento (APM) e alertas tratam erros como sucesso;
- Bibliotecas que lançam exceção em 4xx/5xx (`fetch` com `response.ok` , axios, Cypress `cy.request`) não detectam a falha;
- Respostas de erro podem ser armazenadas em cache como respostas válidas.

SOLUÇÃO PROPOSTA

Centralizar a construção das respostas da API em um único utilitário, que define o status HTTP e mantém o campo `responseCode` no corpo por compatibilidade com os clientes atuais:

```
from django.http import JsonResponse
from django.views.decorators.http import require_http_methods

def api_response(status, **payload):
    return JsonResponse({"responseCode": status, **payload}, status=status)

def method_not_allowed(allowed):
    response = api_response(405, message="This request method is not supported.")
    response["Allow"] = ", ".join(allowed)
    return response

def products_list(request):
    if request.method ≠ "GET":
        return method_not_allowed(["GET"])
    return api_response(200, products=serialize_products())

def search_product(request):
    if request.method ≠ "POST":
        return method_not_allowed(["POST"])
    term = request.POST.get("search_product")
    if not term:
        return api_response(
            400, message="Bad request, search_product parameter is missing in POST request."
        )
    return api_response(200, products=search(term))
```

Compatibilidade: o corpo das respostas não muda; muda apenas o status HTTP. Por isso, a correção deve ser anunciada na página `/api_list` , deixando claro que “Response Code” passa a valer também como status HTTP. Clientes que usam `failOnStatusCode` ou equivalente precisarão tratar respostas 4xx.

CRITÉRIO DE ACEITE

Para todos os casos da tabela de evidências, o status HTTP é igual ao `responseCode` do corpo, e as respostas 405 incluem o cabeçalho `Allow` .

BUG-05 — API serve JSON com Content-Type text/html

Severidade / Prioridade	**Baixa** / P3
Componente	Endpoints `/api/*`
Reprodutibilidade	100%

PASSOS PARA REPRODUZIR

```
curl -s -D - -o /dev/null https://www.automationexercise.com/api/productsList
curl -s -I https://www.automationexercise.com/api/productsList
```

RESULTADO ESPERADO

`Content-Type: application/json` em todas as respostas da API.

RESULTADO OBTIDO

- GET e POST: `content-type: text/html; charset=utf-8`, com corpo JSON;
- HEAD da mesma URL: `content-type: application/json`. O tipo informado no HEAD diverge do GET.

Cypress: `expected 'text/html; charset=utf-8' to include 'application/json'`.

CAUSA RAIZ (HIPÓTESE)

O comportamento é compatível com respostas montadas como `HttpResponse(json.dumps(...))`, que usa `text/html` por padrão no Django. A divergência no HEAD sugere que esse método é tratado por outro caminho (outra camada da aplicação ou da infraestrutura). Como o código do servidor não está disponível, essa causa precisa ser confirmada pela equipe de desenvolvimento.

IMPACTO

Clientes que dependem do `Content-Type` não convertem o corpo automaticamente em JSON (por exemplo, no Cypress, `res.body` chega como texto). O cabeçalho `X-Content-Type-Options: nosniff`, também presente, impede que o navegador corrija o tipo. Além disso, a resposta fica mais exposta a interpretação como HTML.

SOLUÇÃO PROPOSTA

Usar `JsonResponse` em todas as respostas da API. O utilitário `api_response` proposto no BUG-04 já faz isso, então os dois defeitos podem ser corrigidos na mesma entrega. Recomenda-se também confirmar que GET e HEAD passam pela mesma view, para que retornem cabeçalhos idênticos.

CRITÉRIO DE ACEITE

GET, POST e HEAD retornam `Content-Type: application/json`; teste Cypress do BUG-05 passando.

BUG-06 — Conta é excluída por um GET sem confirmação nem token CSRF

Severidade / Prioridade	**Alta** / P1
Componente	Rota `GET /delete_account` (link *Delete Account* do menu)
Reprodutibilidade	100% (2 de 2, com contas diferentes)
Pré-condição	Usuário logado

PASSOS PARA REPRODUZIR

1. Entrar com uma conta de teste em `/login`.
2. Abrir `https://www.automationexercise.com/delete_account` diretamente (barra de endereço ou qualquer link).

RESULTADO ESPERADO

A exclusão pede confirmação explícita e só acontece por `POST` com token CSRF.

RESULTADO OBTIDO

A página exibe "Account Deleted!" e a conta deixa de existir, sem nenhuma confirmação.

EVIDÊNCIAS

- **REA:** cenário login → `goto /delete_account`; árvore de acessibilidade final com `heading "Account Deleted!"`, nas duas execuções;
- **API:** depois do cenário, `POST /api/verifyLogin` com as credenciais da conta responde `{"responseCode": 404, "message": "User not found!"}`;
- **Cypress:** `a conta deve continuar existindo: expected 404 to equal 200`.

ANÁLISE DE CAUSA RAIZ

Uma operação destrutiva está exposta como `GET`, sem etapa de confirmação. O site já usa o token `csrfmiddlewaretoken` do Django nos formulários, mas essa proteção só vale para métodos que alteram estado (`POST`, `PUT`, `DELETE`); um `GET` passa direto.

IMPACTO

- Qualquer página, e-mail ou mensagem com um link para `/delete_account` apaga a conta de quem estiver logado e clicar nele. O cookie de sessão costuma acompanhar uma navegação vinda de outro site; esse cenário entre origens é inferência e não foi testado;
- Pré-carregamento do navegador, extensões ou rastreadores podem disparar a exclusão sem ação do usuário;
- Perda irreversível da conta, do endereço e do histórico do cliente.

SOLUÇÃO PROPOSTA

```
from django.contrib.auth.decorators import login_required
from django.shortcuts import redirect, render
from django.views.decorators.http import require_http_methods

@login_required
@require_http_methods(["GET", "POST"])
def delete_account(request):
    if request.method == "GET":
        # só exibe a tela de confirmação, com o formulário protegido por {% csrf_token %}
        return render(request, "confirm_delete_account.html")
    request.user.delete()
    return redirect("account_deleted")
```

O link do menu passa a abrir a tela de confirmação, e a exclusão só ocorre ao enviar o formulário. Recomenda-se também pedir a senha atual antes de excluir.

CRITÉRIO DE ACEITE

- `GET /delete_account` não altera a conta; só exibe a confirmação;
- `POST` sem token CSRF válido é rejeitado com 403;
- O teste Cypress "[BUG-06] GET /delete_account não deve excluir a conta sem confirmação" passa.

BUG-07 — API expõe dados pessoais sem autenticação

Severidade / Prioridade	**Alta** / P1
Componente	`GET /api/getUserDetailByEmail` (API 14 da lista pública)
Reprodutibilidade	100% (2 de 2, com contas diferentes)
Pré-condição	Conhecer o e-mail de uma conta

PASSOS PARA REPRODUZIR

```
curl "https://www.automationexercise.com/api/getUserDetailByEmail?email=<e-mail da conta>"
```

RESULTADO ESPERADO

A API exige autenticação e só devolve os dados do próprio usuário autenticado.

RESULTADO OBTIDO

Sem nenhuma credencial, a resposta traz `id`, `name`, `email`, `title`, `birth_day`, `birth_month`, `birth_year`, `first_name`, `last_name`, `company`, `address1`, `address2`, `country`, `state`, `city` e `zipcode`.

EVIDÊNCIAS

- **cURL:** duas contas de teste criadas para isso; nas duas, HTTP 200 com o objeto `user` completo;
- **Cypress:** `dados pessoais na resposta sem autenticação: expected { Object (responseCode, user) } to not have property 'user'`.

ANÁLISE DE CAUSA RAIZ

O endpoint procura a conta pelo e-mail recebido na query string e devolve o registro, sem verificar quem faz a requisição. Corresponde a *API3:2023 Broken Object Property Level Authorization* do OWASP API Security Top 10.

IMPACTO

- Exposição de dados pessoais (nome, endereço, data de nascimento) de qualquer cliente cujo e-mail seja conhecido;
- Enumeração de contas: a resposta revela se um e-mail está cadastrado, o que facilita phishing e ataques de senha;
- Risco de não conformidade com leis de proteção de dados (LGPD, GDPR).

SOLUÇÃO PROPOSTA

- Exigir autenticação e responder com os dados do usuário da sessão ou do token, ignorando o e-mail informado;
- Devolver só os campos necessários ao cliente da API;
- Responder da mesma forma para e-mail existente e inexistente quando não houver autorização, para não permitir enumeração.

> O endpoint é documentado na lista pública de APIs do site, e o teste da suíte principal `account.api.cy.js` o usa como contrato. Se a correção for aplicada, aquele teste deverá ser ajustado junto.

CRITÉRIO DE ACEITE

Sem autenticação, a resposta não contém dados pessoais (HTTP 401); teste Cypress do BUG-07 passando.

BUG-08 — Contact Us confirma o envio, mas não envia nada

Severidade / Prioridade	**Alta** / P1
Componente	`/contact_us`, script embutido na página
Reprodutibilidade	100% (3 de 3)
Pré-condição	Nenhuma

PASSOS PARA REPRODUZIR

1. Acessar `/contact_us` e preencher nome, e-mail, assunto e mensagem.
2. Clicar em *Submit* e confirmar o diálogo "Press OK to proceed!" com *OK*.

RESULTADO ESPERADO

A mensagem é enviada ao servidor (`POST /contact_us`) e só então a confirmação aparece.

RESULTADO OBTIDO

Aparece "Success! Your details have been submitted successfully.", mas nenhuma requisição sai para o site.

EVIDÊNCIAS

- **Playwright (diálogo aceito):** duas execuções com `requisicoes_apos_envio: []` para o domínio do site; numa terceira, registrando todas as origens, só houve tráfego de anúncios do Google;
- **REA:** com o diálogo recusado (comportamento padrão), também nenhuma requisição;
- **Cypress:** `requisições enviadas ao site: expected [] to have a length above 0 but got 0`.

ANÁLISE DE CAUSA RAIZ

O script da página intercepta o envio e retorna `false` nos dois ramos do `confirm()`:

```
$("#contact-us-form").on("submit", function() {
    var r = confirm("Press OK to proceed!");
    if (r == true) {
        $(".alert-success").html("Success! Your details have been submitted successfully.");
        $(".alert-success").attr("style", "display: block;");
        $("#form-section").html('<a class="btn btn-success" href="/">...Home</a>');
        return false;
    } else {
        return false;
    }
});
```

Além disso, o `<form>` não declara `enctype="multipart/form-data"`: mesmo que o envio acontecesse, o arquivo anexado no campo `upload_file` não seria transmitido.

IMPACTO

- Pedidos de suporte, reclamações e solicitações se perdem sem que cliente ou empresa percebam;
- A mensagem de sucesso induz o cliente a esperar uma resposta que nunca virá.

SOLUÇÃO PROPOSTA

- Deixar o formulário ser enviado (retornar `true` no ramo confirmado, ou enviar por AJAX) e exibir a confirmação só depois da resposta de sucesso do servidor;
- Declarar `enctype="multipart/form-data"` e validar o arquivo no back-end (tipo e tamanho);
- Tornar obrigatórios os campos de mensagem e assunto.

CRITÉRIO DE ACEITE

O envio gera `POST /contact_us` com os campos e o anexo; a confirmação só aparece após resposta 2xx; teste Cypress do BUG-08 passando.

BUG-09 — Quantidade vazia ou não inteira gera erro 500

Severidade / Prioridade	**Média** / P2
Componente	`/static/js/cart.js`, endpoint `GET /add_to_cart/{id}`
Reprodutibilidade	100% (2 de 2 em cada valor)
Pré-condição	Nenhuma

PASSOS PARA REPRODUZIR

1. Acessar `/product_details/5`.
2. Apagar o conteúdo do campo *Quantity* (ou digitar `2.5` ou `1e2`, que o campo numérico aceita).
3. Clicar em *Add to cart*.

RESULTADO ESPERADO

Mensagem de validação na tela; nenhum erro no servidor.

RESULTADO OBTIDO

O servidor responde **HTTP 500** com a página de erro do Django em modo de depuração. A tela não abre o modal "Added!" nem mostra erro, e o carrinho continua vazio.

EVIDÊNCIAS

Valor digitado	Requisição	Status
(vazio)	`GET /add_to_cart/5?quantity=`	500
`2.5`	`GET /add_to_cart/5?quantity=2.5`	500
`1e2`	`GET /add_to_cart/5?quantity=1e2`	500

- **REA:** rede com as respostas 500 acima; console com `Failed to load resource: the server responded with a status of 500`; árvore de acessibilidade sem o modal e carrinho com "Cart is empty!";
- **Resposta do servidor:** página de erro com `DEBUG = True` e traceback de `ValueError`;
- **Cypress:** `status HTTP de /add_to_cart: expected 500 to be below 500`.

ANÁLISE DE CAUSA RAIZ

É a mesma ausência de validação do BUG-01, agora no tipo do valor. O campo não declara `step`, então o navegador aceita texto vazio, decimais e notação científica; o `cart.js` envia o texto bruto; e o servidor converte com `int()` sem tratar a exceção. Como o Django está com `DEBUG` ligado em produção, o erro devolve o traceback completo.

IMPACTO

- O produto não entra no carrinho e o usuário não recebe nenhum aviso;
- A página de depuração expõe detalhes internos da aplicação a qualquer visitante;
- Erros 500 recorrentes poluem a monitoração e os indicadores de disponibilidade.

SOLUÇÃO PROPOSTA

A validação do BUG-01 (converter com tratamento de exceção e responder 400) resolve este defeito. Também é necessário configurar `DEBUG = False` em produção e declarar `step="1"` no campo, para que a validação do navegador rejeite valores não inteiros.

CRITÉRIO DE ACEITE

Os três valores da tabela retornam 400 com mensagem de validação, sem página de depuração; testes Cypress do BUG-09 passando.

BUG-10 — Assinatura da newsletter não é registrada

Severidade / Prioridade	**Média** / P2
Componente	Rodapé de todas as páginas, `/static/js/subscription.js`
Reprodutibilidade	100% (2 de 2)

PASSOS PARA REPRODUZIR

1. No rodapé de qualquer página, informar um e-mail em *Subscription*.
2. Clicar na seta de envio.

RESULTADO ESPERADO

O e-mail é enviado ao servidor e a assinatura é registrada.

RESULTADO OBTIDO

Aparece “You have been successfully subscribed!”, mas nenhuma requisição é feita ao site.

EVIDÊNCIAS

- **REA:** nenhuma requisição ao domínio do site depois do clique, nas duas execuções;
- **Cypress:** `requisições enviadas ao site: expected [] to have a length above 0 but got 0`.

ANÁLISE DE CAUSA RAIZ

```
$(".searchform").on("submit", function() {
    $("#success-subscribe").removeClass('hide');
    setTimeout(function(){
        $("#success-subscribe").addClass('hide');
        document.getElementById("susbscribe_email").value = '';
    }, 1500);
    return false;
});
```

O script mostra a mensagem, limpa o campo e cancela o envio. O formulário até contém o `csrfmiddlewaretoken`, mas nunca é submetido.

IMPACTO

Assinantes não recebem as comunicações prometidas e a base de contatos de marketing não cresce.

SOLUÇÃO PROPOSTA

Enviar o e-mail por `POST` (com o token CSRF já presente), validar e gravar no back-end, e mostrar a confirmação só depois da resposta de sucesso. O ideal é usar confirmação por e-mail (*double opt-in*).

CRITÉRIO DE ACEITE

A assinatura gera uma requisição ao servidor e a mensagem depende da resposta; teste Cypress do BUG-10 passando.

BUG-11 — Avaliação de produto não é enviada

Severidade / Prioridade	**Média** / P2
Componente	`/product_details/{id}`, aba *Write Your Review*, script embutido na página
Reprodutibilidade	100% (2 de 2)

PASSOS PARA REPRODUZIR

1. Acessar `/product_details/1`.
2. Preencher nome, e-mail e texto em *Write Your Review* e clicar em *Submit*.

RESULTADO ESPERADO

A avaliação é enviada ao servidor e passa a aparecer na página do produto (ou fica pendente de moderação, com aviso).

RESULTADO OBTIDO

Aparece "Thank you for your review.", mas nenhuma requisição é feita ao site e a avaliação não aparece em lugar nenhum.

EVIDÊNCIAS

- **REA:** nenhuma requisição ao domínio do site depois do envio, nas duas execuções;
- **Cypress:** `requisições enviadas ao site: expected [] to have a length above 0 but got 0`.

ANÁLISE DE CAUSA RAIZ

Assim como no BUG-10, o script embutido trata o `submit` de `#review-form` apenas mostrando e escondendo a mensagem de agradecimento, e o formulário não tem `action` nem é enviado.

IMPACTO

Clientes perdem tempo escrevendo avaliações que somem, e a loja não recebe o retorno nem a prova social das avaliações.

SOLUÇÃO PROPOSTA

Enviar a avaliação por `POST` com token CSRF, gravá-la (com moderação, se necessário) e exibir a confirmação só após a resposta do servidor.

CRITÉRIO DE ACEITE

O envio gera uma requisição ao servidor e a avaliação fica disponível ou pendente de moderação; teste Cypress do BUG-11 passando.

4. Plano de correção recomendado

Entrega	Defeitos	Ações	Esforço estimado
1 — Urgente	BUG-06, BUG-07	Exclusão de conta só por `POST` com CSRF e confirmação; autenticação no endpoint de dados do usuário	Baixo a médio
2 — Urgente	BUG-01, BUG-02, BUG-09	Validação de quantidade no back-end (inteiro, 1..99, resposta 400), restrição no banco, validação no front-end, `DEBUG = False` e saneamento de dados	Baixo a médio
3	BUG-08, BUG-10, BUG-11	Envio real dos três formulários, com persistência e confirmação só após a resposta do servidor	Médio
4	BUG-04, BUG-05	Utilitário único de resposta da API com status e Content-Type corretos; atualização da documentação	Baixo
5	BUG-03	Fontes via HTTPS no `<head>`; cabeçalho CSP `upgrade-insecure-requests`	Muito baixo

> As estimativas são qualitativas e baseadas apenas no comportamento observado. O código do servidor não foi analisado diretamente.

5. Recomendações adicionais

- **Operações que alteram estado via GET:** além do `/delete_account` (BUG-06), `/add_to_cart/{id}` e `/delete_cart/{id}` alteram o carrinho por meio de requisições GET. Recomenda-se migrá-las para POST/DELETE com proteção CSRF, porque requisições GET podem ser disparadas por links, pré-carregamento do navegador ou rastreadores.
- **Mensagens de sucesso:** exibir confirmações só depois da resposta do servidor, nunca antes (origem comum dos BUG-08, BUG-10 e BUG-11).
- **Configuração de produção:** desligar `DEBUG` no Django; hoje as páginas de erro 404 e 500 expõem rotas internas e tracebacks.
- **Regressão automatizada:** os testes Cypress deste relatório estão em `cypress/e2e/known-bugs/` do repositório e rodam diariamente no pipeline de integração contínua, em um job separado da suíte principal. Eles falham hoje e devem passar após as correções.
- **Monitoramento de console:** adicionar verificação automática de erros de console (Mixed Content, exceções JavaScript) e de respostas 5xx nos testes de fumaça.
- **Contrato da API:** publicar uma especificação OpenAPI com status HTTP, cabeçalhos, autenticação e esquemas de resposta para cada endpoint.

6. Rastreabilidade com os testes automatizados

Execução de 16/10/2026: `npm run test:bugs`, 17 testes, 17 falhas, todas pelas asserções que descrevem os defeitos.

Defeito	Teste Cypress	Resultado atual
BUG-01	[BUG-01] não deve aceitar quantidade negativa (-3)	Falha — `expected -3 to be above 0`
BUG-02	[BUG-02] não deve aceitar quantidade zero (0)	Falha — `expected 0 to be above 0`
BUG-03	[BUG-03] as folhas de estilo não devem importar recursos via http://	Falha — 3 imports inseguros em `main.css`
BUG-04	[BUG-04] POST /api/productsList deve retornar HTTP 405	Falha — `expected 200 to equal 405`
BUG-04	[BUG-04] PUT /api/brandsList deve retornar HTTP 405	Falha — `expected 200 to equal 405`
BUG-04	[BUG-04] DELETE /api/verifyLogin deve retornar HTTP 405	Falha — `expected 200 to equal 405`
BUG-04	[BUG-04] POST /api/searchProduct sem parâmetro deve retornar HTTP 400	Falha — `expected 200 to equal 400`
BUG-04	[BUG-04] POST /api/verifyLogin com parâmetros na query string deve retornar HTTP 400	Falha — `expected 200 to equal 400`
BUG-05	[BUG-05] GET /api/productsList deve declarar Content-Type JSON	Falha — `expected 'text/html; charset=utf-8' to include 'application/json'`
BUG-06	[BUG-06] GET /delete_account não deve excluir a conta sem confirmação	Falha — `expected 404 to equal 200`
BUG-07	[BUG-07] GET /api/getUserDetailByEmail não deve expor dados pessoais sem autenticação	Falha — resposta contém `user`
BUG-08	[BUG-08] Contact Us deve enviar a mensagem ao servidor	Falha — `expected [] to have a length above 0`
BUG-09	[BUG-09] campo vazio não deve gerar erro 500	Falha — `expected 500 to be below 500`
BUG-09	[BUG-09] decimal (2.5) não deve gerar erro 500	Falha — `expected 500 to be below 500`
BUG-09	[BUG-09] notação científica (1e2) não deve gerar erro 500	Falha — `expected 500 to be below 500`
BUG-10	[BUG-10] assinatura da newsletter deve ser enviada ao servidor	Falha — `expected [] to have a length above 0`
BUG-11	[BUG-11] avaliação de produto deve ser enviada ao servidor	Falha — `expected [] to have a length above 0`

7. Limitações

- A análise foi feita de fora (caixa-preta e caixa-cinza): as causas raiz no servidor são inferidas a partir do comportamento observado, do HTML, do CSS e do JavaScript públicos. Os trechos de código do back-end são propostas ilustrativas, e não cópias do código existente.
- Os resultados refletem o estado do site em 14/10/2026 (BUG-01 a BUG-05) e 16/10/2026 (BUG-06 a BUG-11, e reteste de todos). Por ser um ambiente público e compartilhado, o comportamento pode mudar sem aviso.
- O Automation Exercise é um site de prática de automação; parte dos comportamentos (como os formulários sem envio) pode ser intencional. Eles foram registrados porque, do ponto de vista do usuário, são defeitos.
- O ataque entre origens do BUG-06 é inferido, não demonstrado. A página de depuração do BUG-09 não foi analisada além do necessário para identificar o erro.
- As áreas listadas como fora do escopo (seção 2.1) não foram avaliadas, e a ausência de defeitos nelas não está comprovada.